TEORIA DE NUMEROS

vamos a ver conceptos necesarios

número primo es un número natural mayor que 1 que tiene únicamente dos divisores positivos distintos

que es un divisor Sea a y b enteros, se dice que a|b (a divide a b) si es que existe un entero x tal que a*x = b.

cómo implementar solución ingenua

```
for (int i = 2; i < n; i++)
{
    if (n % i == 0)
    {
        res = true;
        break;
    }
}
if (res==false)
    cout << "positivo es primo";
else
    cout << "Nega";
return 0;
```

ahora nos fijamos en este detalle

lo que nos lleva a esta solución

```
int main()
{
    ios::sync_with_stdio(0);
    cin.tie(0);
    cout.tie(0);
    int n;
    cin >> n;
    bool res = false;
    for (int i = 2; i*i <=n; i++)
    {
        if (n % i == 0)
        {
            res = true;
            break;
        }
    }
    if (res==false)
        cout << "positivo es primo";
    else
        cout << "Nega";
    return 0;
}
```

Teorema fundamental de la aritmética

para cualquier número mayor a uno hay una factorización con números primos

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k},$$

ahora con una idea similar al anterior vamos a factorizar en sus valores primos

```
    map <int, int> factores;
    for (int i = 2; i*i <=n; i++)
    {
        while (n % i == 0)
        {
            factores[i]++;
            n /= i;
        }
    }
    if (n > 1) factores[n]++;
    for (auto it : factores)
    {
        cout << it.first << " " << it.second << endl;
    }

    return 0;
}
```

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k},$$

con esto se puede calcular el número de divisores a partir de esta fórmula

$$\tau(n) = \prod_{i=1}^{k} (\alpha_i + 1),$$

$$n = p_1^{\alpha_1} p_2^{\alpha_2} \cdots p_k^{\alpha_k},$$

suma de divisores

$$\sigma(n) = \prod_{i=1}^{k} (1 + p_i + \cdots + p_i^{\alpha_i}) = \prod_{i=1}^{k} \frac{p_i^{\alpha_i+1} - 1}{p_i - 1},$$

Criba de Eratóstenes

276 a. C.[1]-Alejandría,

Basicamente es encontrar los numeros primos de uno hasta en en

complejidad en O(nLog(n)) hay criba mejorado que es en nLog(log(n))

incluso Lineal pero por ahora vamos a ver nomas O(nLog(n)

2	3	4	5	6	7	8	9	10	11	12	13	14	15	16

2	3	~~4~~	5	~~6~~	7	~~8~~	9	~~10~~	11	~~12~~	13	~~14~~	15	~~16~~

2	**3**	~~4~~	5	~~6~~	7	~~8~~	~~9~~	~~10~~	11	~~12~~	13	~~14~~	~~15~~	~~16~~

2	**3**	~~4~~	**5**	~~6~~	7	~~8~~	~~9~~	~~10~~	11	~~12~~	13	~~14~~	~~15~~	~~16~~

2	**3**	~~4~~	**5**	~~6~~	**7**	~~8~~	~~9~~	~~10~~	11	~~12~~	13	~~14~~	~~15~~	~~16~~

2	**3**	~~4~~	**5**	~~6~~	**7**	~~8~~	~~9~~	~~10~~	**11**	~~12~~	13	~~14~~	~~15~~	~~16~~

2	**3**	~~4~~	**5**	~~6~~	**7**	~~8~~	~~9~~	~~10~~	**11**	~~12~~	**13**	~~14~~	~~15~~	~~16~~

ahora como implementar

```
bitset<10000010> criba;
void cribon()
{
    criba.set();
    criba[0] = criba[1] = 0;
    for (int i = 2; i <= 10000000; i++)
    {
        if (criba[i])
        {
            for (int j = 2*i; j <= 10000000; j += i)
            {
                criba[j] = 0;
            }
        }
    }
}
```

ahora podemos mejorarlo alguno tiene alguna idea

pista cómo mejoramos en la primalidad

```
bitset<10000010> criba;
void cribon()
{
    criba.set();
    criba[0] = criba[1] = 0;
    for (int i = 2; i<= 10000000; i++)
    {
        if (criba[i])
        {
            for (int j = 2*i; j <= 10000000; j += i)
            {
                criba[j] = 0;
            }
        }
    }
}
```

Ahora veremos una función de la criba

se puede factorizar posterior a la criba factorizar en log(n)

```
bitset<10000010> criba;
vector<int> primos;
void cribon()
{
    criba.set();
    criba[0] = criba[1] = 0;
    for (int i = 2; i *i<= 10000000; i++)
    {
        if (criba[i])
        {
            primos.push_back(i);
            for (int j = 2*i; j*j<= 10000000; j += i)
            {
                criba[j] = 0;
            }

        }
    }
}
```

```
cribon();
map <int, int> factores;
//numeros primos antes de n es igual n/l

int num=n/ log(n);
for (int i = 0; i < primos.size(); i++)
{
    while (n % primos[i] == 0)
    {
        factores[primos[i]]++;
        n /= primos[i];
    }
}
return 0;
```

GCD y LCM

Greatest Common Divisor (Máximo Común Divisor)

Dados dos números a y b, su GCD es el mayor número que divide a ambos

Least Common Multiple (Mínimo Común Múltiplo)

Dados dos números a y b, su LCM es el menor número que es múltiplo de ambos

$$\gcd(2^3 \cdot 3^2 \cdot 5^1, 2^2 \cdot 3^4 \cdot 7^1) = 2^2 \cdot 3^2$$

$$\operatorname{lcm}(2^3 \cdot 3^2 \cdot 5^1, 2^2 \cdot 3^4 \cdot 7^1) = 2^3 \cdot 3^4 \cdot 5^1 \cdot 7^1$$

implementación

```
int a, b;
cin >> a >> b;
cout<<__gcd(a, b)<<endl;
```

```
int gcd(int a, int b) {
    if (b == 0) return a;
    return gcd(b, a%b);
}
```

un pequeño paso por euclides extendido(diofanticas)

$$ax + by = \gcd(a, b).$$

```
tuple<int,int,int> euclid(int a, int b) {
    if (b == 0) {
        return {1,0,a};
    } else {
        int x,y,g;
        tie(x,y,g) = euclid(b,a%b);
        return {y,x-(a/b)*y,g};
    }
}
```

Aritmética Modular

- Recordemos que $\frac{a}{m} \rightarrow a = m \cdot c + r$, siendo c el cociente y r el residuo
- La aritmética modular trabaja solo con los residuos
- $a \mod m \rightarrow$ residuo de dividir a entre m

congruencia

Dos números son congruentes módulo n si tienen el mismo residuo al dividir por n. Se denota como $a \equiv b \pmod{n}$.

operaciones

Suma =(a+b)%m=((a%m)+(b%m))%m

Resta=(a-b)%m=((a%m)-(b%m)+m)%m

multiplicación=(a*b)%m=((a%m)*(b%m))%m

demostremos la multiplicación

$$((m \cdot c_a + r_a) \cdot (m \cdot c_b + r_b))\%m$$

$$(m^2 \cdot c_a \cdot c_b + m \cdot c_a \cdot r_b + m \cdot c_b \cdot r_a + r_a \cdot r_b)\%m$$

Repartimos el módulo en la suma

$$((m^2 \cdot c_a \cdot c_b\%m) + (m \cdot c_a \cdot r_b\%m) + (m \cdot c_b \cdot r_a\%m) + (r_a \cdot r_b\%m))\%m$$

$$(0 + 0 + 0 + (r_a \cdot r_b\%m))\%m$$

$$r_x = x\%m \rightarrow (a\%m \cdot b\%m)\%m$$

Porque

no se puede representar la división como una suma

División

ahora viene lo interesante

$$(a/b)\%m = (a\%m \cdot b^{-1}\%m)\%m$$

$a^{-1} \mod m$ es el inverso modular de a módulo m

¿Cómo obtenemos $a^{-1} \mod m$?

Pequeño teorema de Fermat: $a^{m-1} \equiv 1 \mod m$, si m es primo y a no es múltiplo de m

$$a^{m-2} \equiv a^{-1} \mod m$$

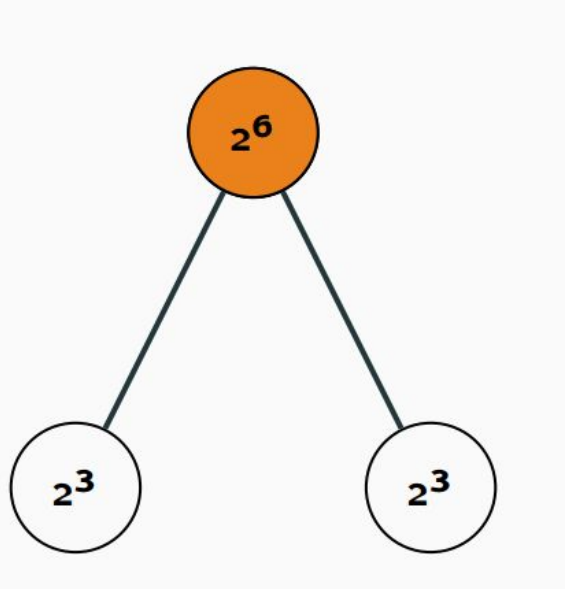
$$(a/b)\%m = (a\%m \cdot b^{m-2}\%m)\%m$$

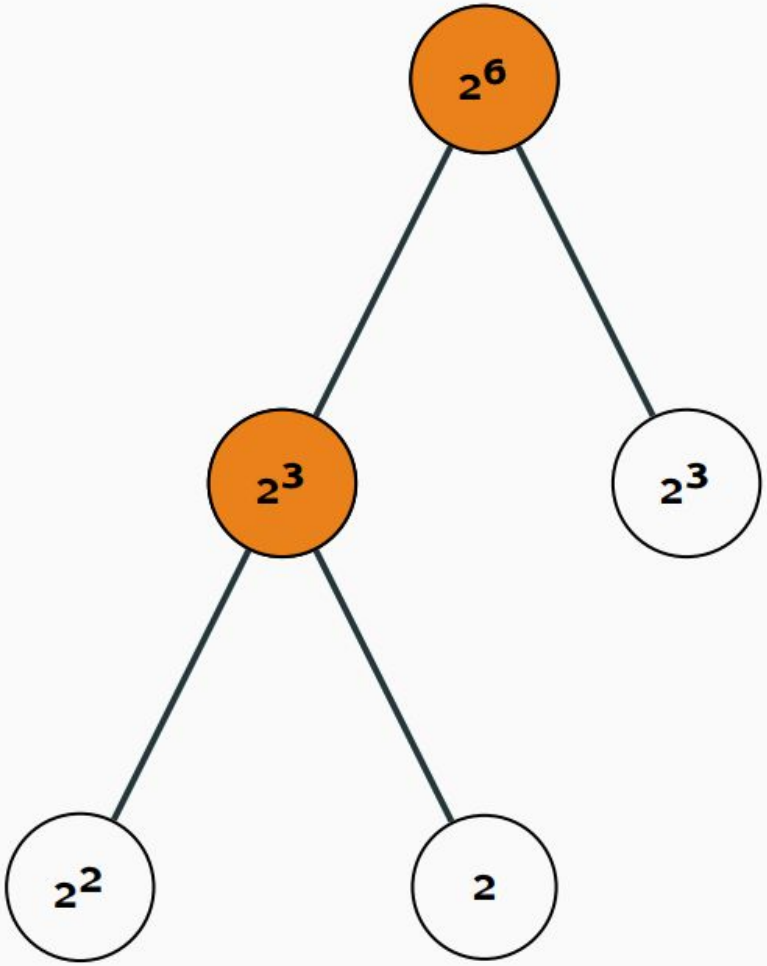
Exponenciación binomial

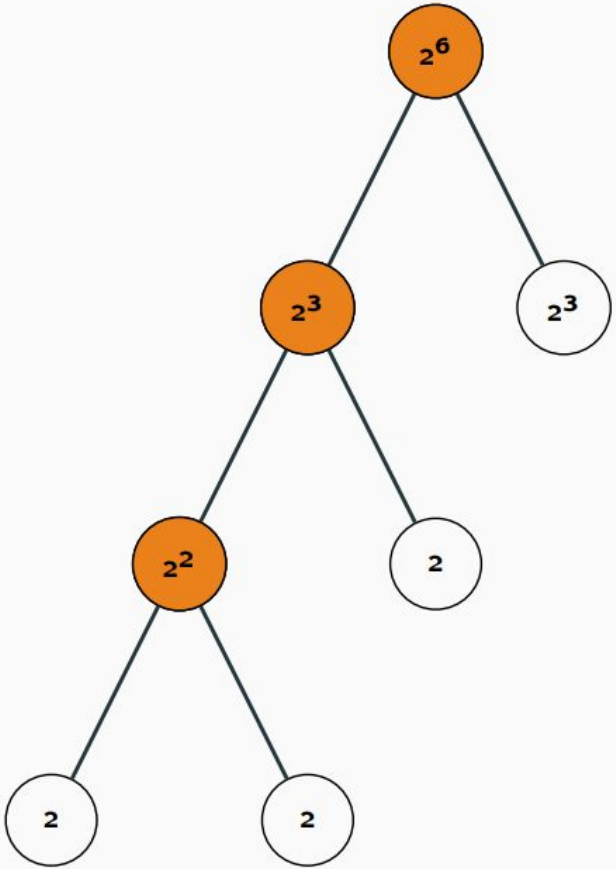
Para calcular $b^{m-2} \mod m$, necesitamos una forma eficiente de calcular potencias

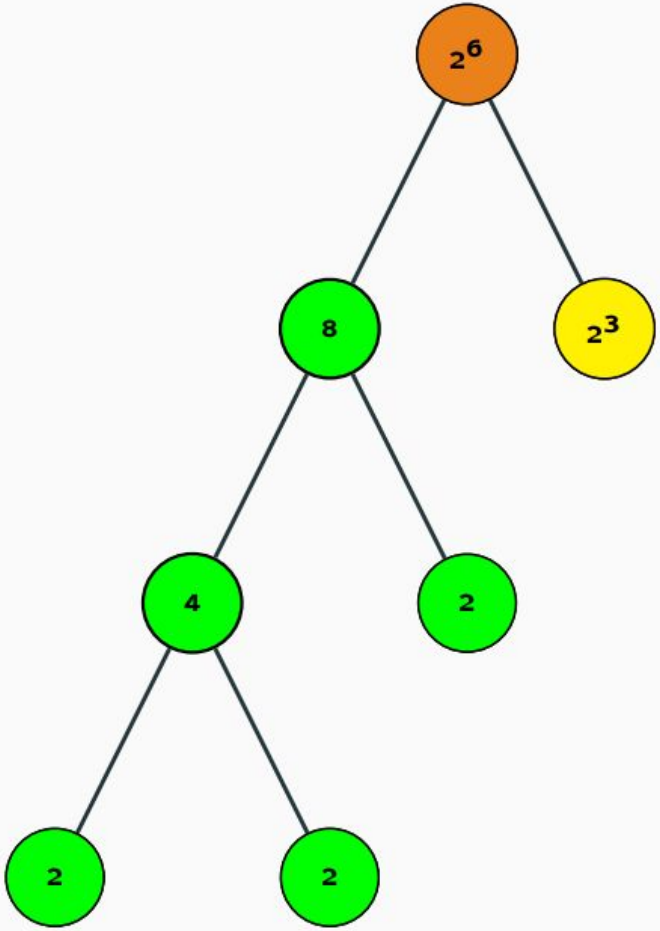
Multiplicar b m veces es $O(m)$, m suele ser un valor cercano a 10^9

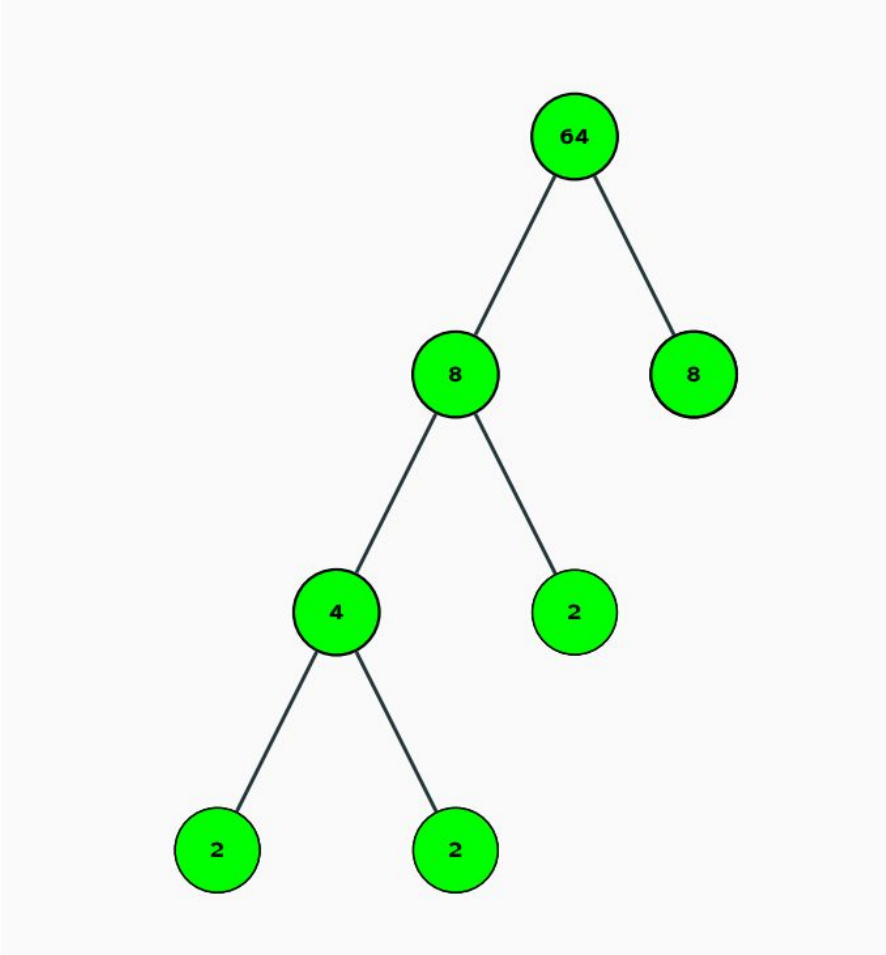
2^6


2^6
2^3
2^3


2^6
2^3
2^3
2^2
2


2^6
2^3
2^3
2^2
2
2
2


2^6
8
2^3
4
2
2
2


64
8
8
4
2
2
2

```
int binPow(int b, int e, int mod) {
    if (e == 0) return 1;
    if (e == 1) return b;
  int res;
  if (e % 2 == 0) {
  res = binPow(b, e / 2);
  res = res * res % mod;
  }
  else {
  res = binPow(b, e - 1) * b % mod;
  }
  return res;
  }
```

Dos números son coprimos si es que el gcd(a, b) = 1.

Función φ de euler (cantidad de números coprimos a n menores a n):

$$\varphi(n) = \prod_{i=1}^{k} p_i^{\alpha_i - 1}(p_i - 1).$$

Teorema de euler:

$$x^{\varphi(m)} \bmod m = 1$$

Funciones multiplicativas

Para demostrar que una función es multiplicativa, es necesario comprobar dos condiciones:

Multiplicidad: Si para dos números primos distintos p y q, y sus respectivos exponentes k y l, se tiene que $f(p^k * q^l) = f(p^k) * f(q^l)$, entonces la función es multiplicativa.

Valor en el número 1: La función debe cumplir $f(1) = 1$, es decir, tiene un valor específico asignado para el número 1.